

EXERCÍCIOS

1- Sistema de Auditoria com Prisma ORM

Contexto

Uma empresa precisa registrar todas as alterações realizadas no sistema para fins de rastreabilidade, segurança e conformidade.

Toda vez que um registro for criado, atualizado ou removido, o sistema deverá gerar automaticamente um log de auditoria contendo:

- Quem realizou a ação.
- Qual entidade foi alterada.
- Qual ação foi executada.
- Os dados antes da alteração.
- Os dados depois da alteração.
- Data e hora da operação.

Objetivo

Implementar um sistema de auditoria utilizando Prisma ORM capaz de registrar:

- CREATE
- UPDATE
- DELETE

para qualquer entidade do sistema.

Cenário

Você está desenvolvendo um sistema de gerenciamento de usuários.

O sistema possui:

- Usuários do sistema
- Logs de auditoria

Sempre que um usuário for criado, atualizado ou excluído, um registro deve ser salvo na tabela de auditoria.

Requisitos Funcionais

RF01 – Registrar Criação

Quando um usuário for criado:

João

email: joao@email.com

Deve ser criado um log:

```
{
  "entity": "User",
  "entityId": "123",
  "action": "CREATE",
  "oldData": null,
  "newData": {
    "name": "João",
    "email": "joao@email.com"
  }
}
```

RF02 – Registrar Atualização

Antes:

```
{
  "name": "João",
  "email": "joao@email.com"
}
```

Depois:

```
{
  "name": "João Silva",
  "email": "joao@email.com"
}
```

Log:

```
{
  "action": "UPDATE",
  "oldData": {
    "name": "João",
    "email": "joao@email.com"
  },
  "newData": {
```

ATIVIDADE PRISMA ORM
CURSO TÉCNICO DESENVOLVIMENTO DE SISTEMAS

```
"name": "João Silva",  
"email": "joao@email.com"  
}  
}
```

RF03 – Registrar Exclusão

Antes:

```
{  
  "name": "João Silva"  
}
```

Depois:

null

Log:

```
{  
  "action": "DELETE",  
  "oldData": {  
    "name": "João Silva"  
  },  
  "newData": null  
}
```

Modelagem Prisma

User

```
model User {  
  id      String  @id @default(uuid())  
  name    String  
  email   String  @unique  
  
  createdAt DateTime @default(now())  
  updatedAt DateTime @updatedAt  
}
```

AuditLog

```
model AuditLog {  
  id      String  @id @default(uuid())  
  
  entity   String  
  entityId String
```

```
action    AuditAction

oldData   Json?
newData   Json?

performedBy String?

createdAt DateTime @default(now())
}
```

Enum

```
enum AuditAction {
    CREATE
    UPDATE
    DELETE
}
```

2- Sistema de Permissões (RBAC) com Prisma ORM

Contexto

Uma empresa está desenvolvendo um sistema administrativo onde diferentes usuários possuem diferentes níveis de acesso.

Você deverá implementar um sistema de RBAC (Role-Based Access Control) utilizando Prisma ORM.

Objetivo

Criar um sistema onde:

- Um usuário pode possuir várias roles (papéis).
- Uma role pode ser atribuída a vários usuários.
- Uma role possui várias permissões.
- Uma permissão pode pertencer a várias roles.
- O sistema deve verificar se um usuário possui determinada permissão.

Requisitos Funcionais

Usuários

O sistema deve permitir:

- Criar usuários.
- Listar usuários.
- Atribuir roles aos usuários.

Exemplos:

ATIVIDADE PRISMA ORM
CURSO TÉCNICO DESENVOLVIMENTO DE SISTEMAS

Usuário	Role
João	ADMIN
Maria	EDITOR
Pedro	VIEWER

Roles

O sistema deve possuir as seguintes roles iniciais:

ADMIN

Permissões:

CREATE_USER
UPDATE_USER
DELETE_USER
VIEW_USER

EDITOR

Permissões:

UPDATE_USER
VIEW_USER

VIEWER

Permissões:

VIEW_USER

Permissões

Permissões disponíveis:

CREATE_USER
UPDATE_USER
DELETE_USER
VIEW_USER
CREATE_POST
UPDATE_POST
DELETE_POST
VIEW_POST